

FOCUS TIMER

MÉTHODE POMODORO

PRÉSENTATION DU PROJET



CAHIER DES CHARGES

FONCTIONNALITÉS

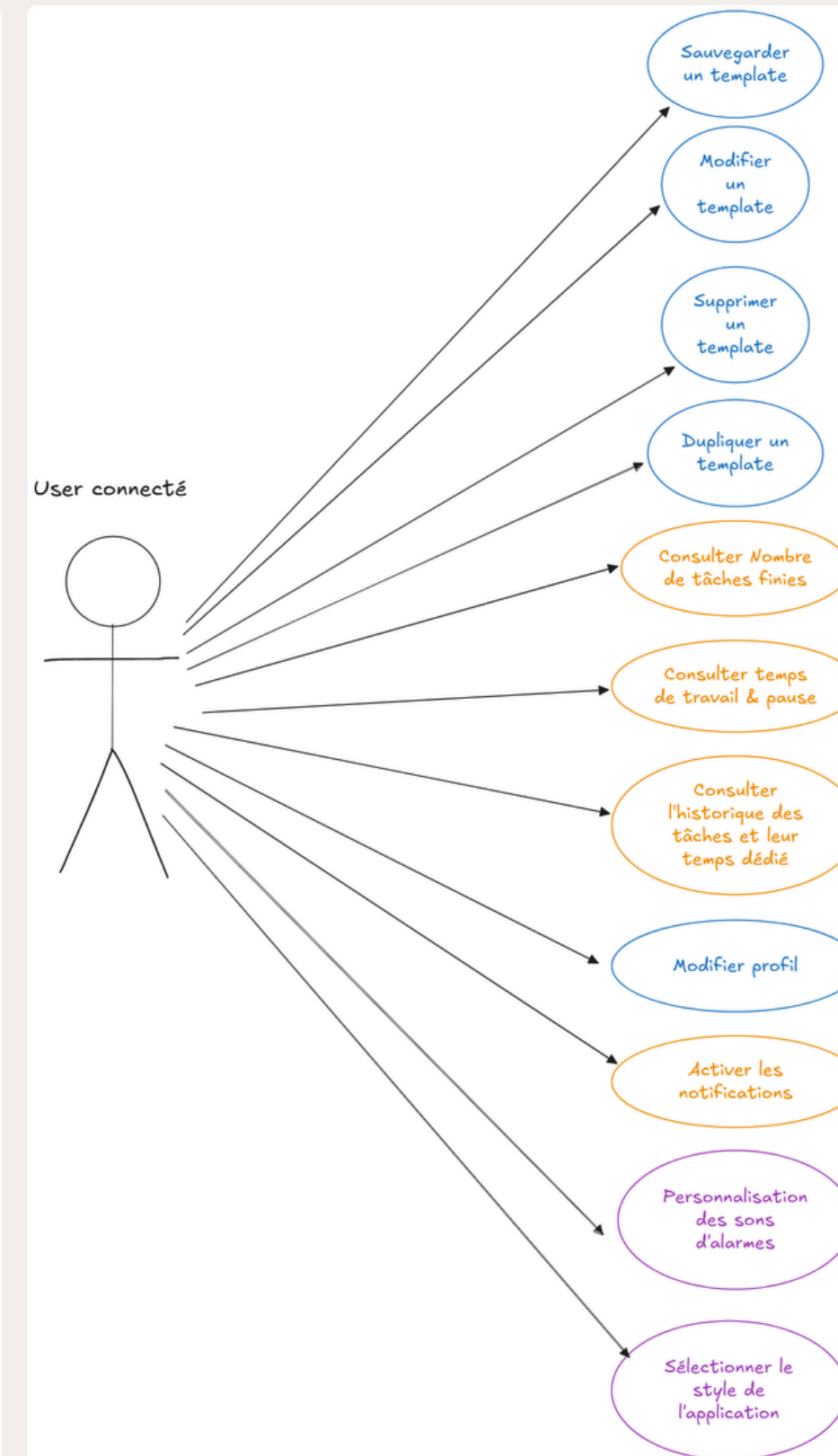
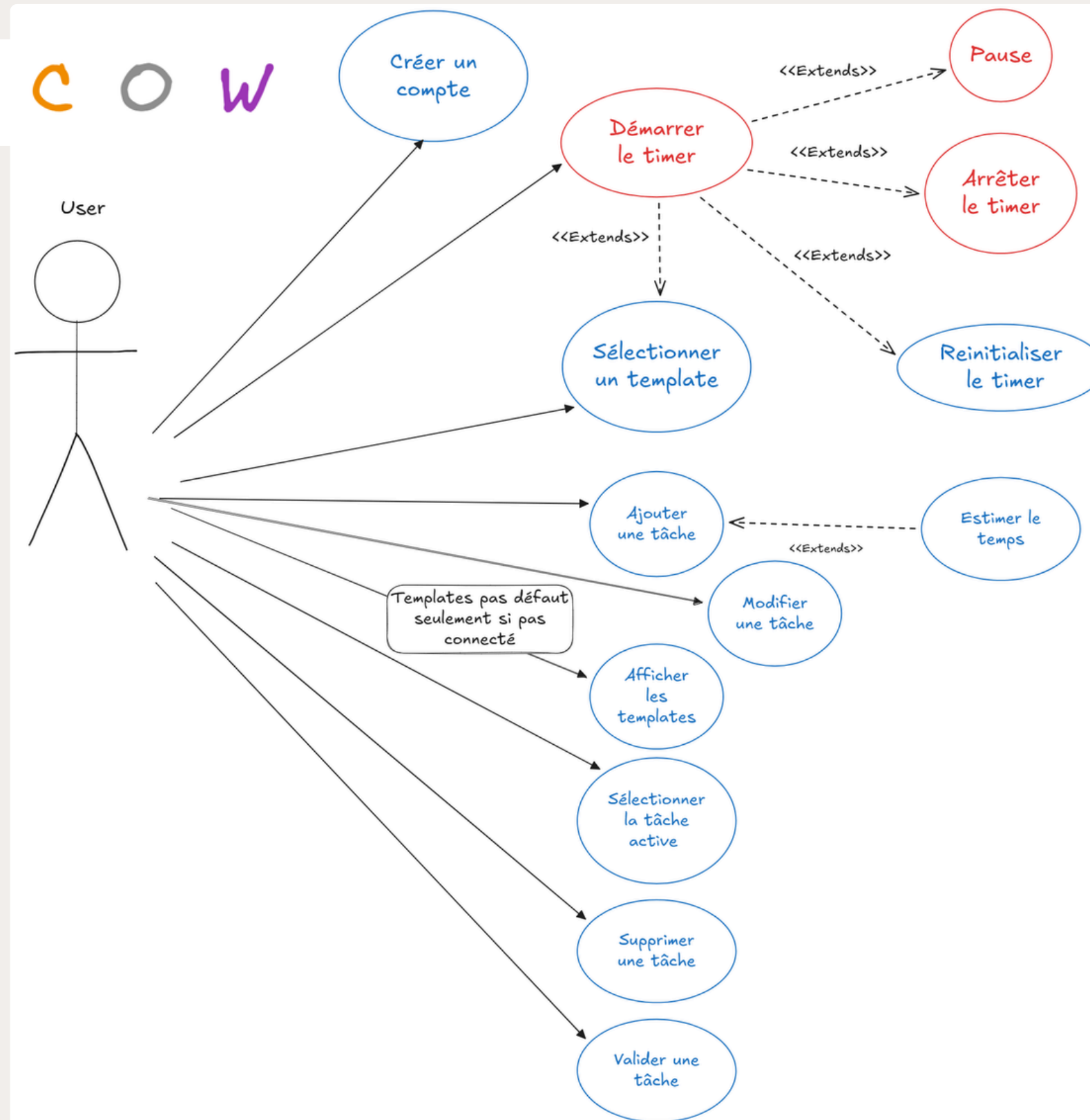
- MINUTEUR
- CYCLE TRAVAIL/PAUSE
- TÂCHES
- PARAMÈTRES
- STATISTIQUES

CONTRAINTES TECHNIQUES

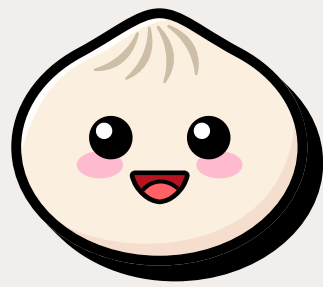
- TWELVE-FACTOR
- MODE OFFLINE/ONLINE
- API REST
- DOCKER

FONCTIONNALITÉS

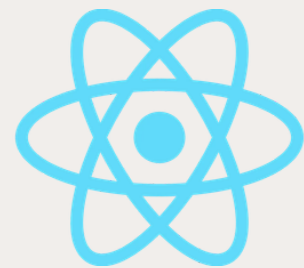
M O S C O W



CHOIX DES TECHNOLOGIES



BUN



REACT



TAILWIND



DOCKER



ELYSIAJS

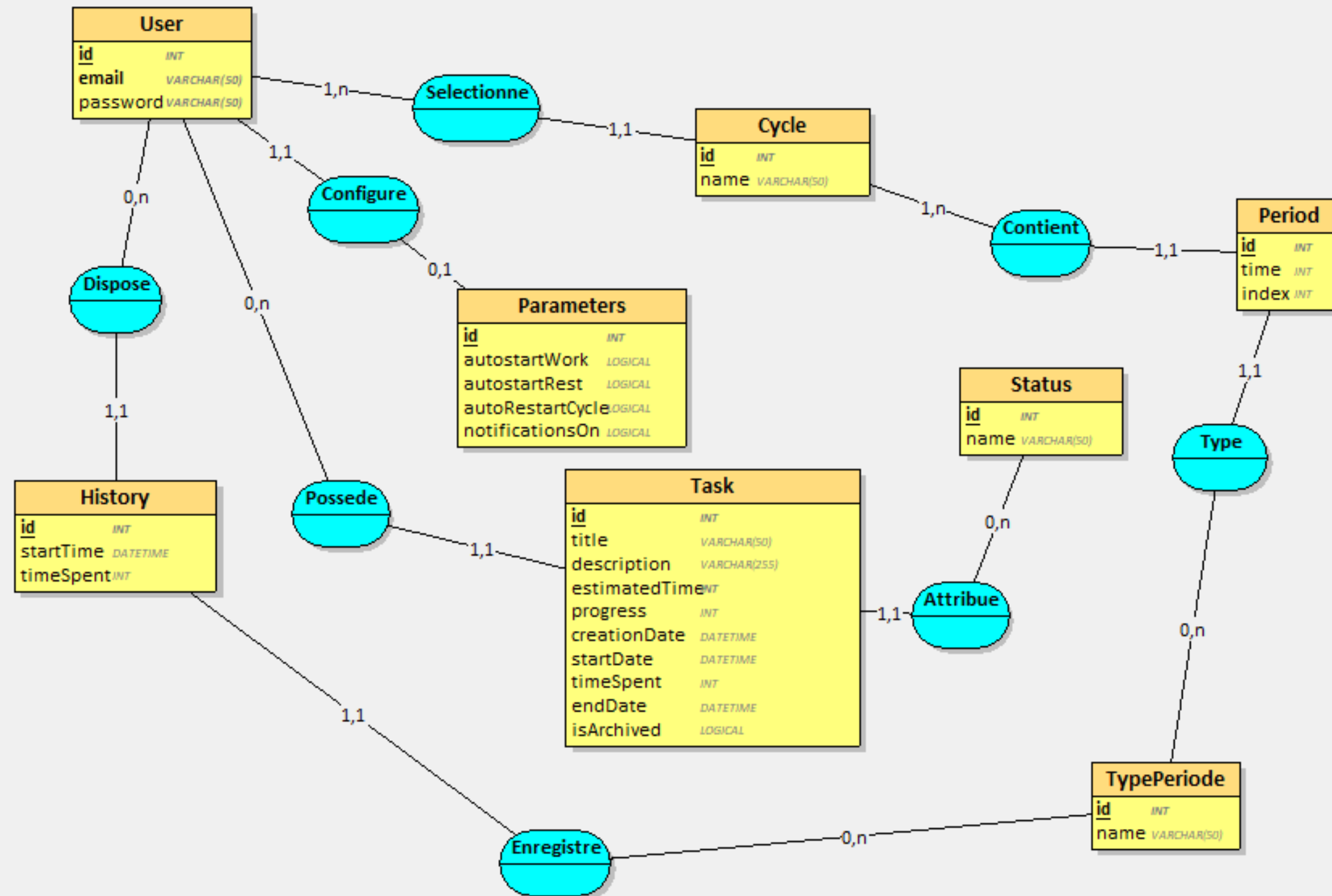
12 PRINCIPES

	PRINCIPE	
1	Codebase	1 repo
2	Dependencies	package.json
3	Configuration	.env
4	Backing services	Backend versatile
5	Build, release, run	CI/CD existant
6	Processes	NON
7	Port binding	App sur des ports
8	Concurrency	-
9	Disposability	Bun + docker rapide
10	Dev/prod parity	docker utilisé
11	Logs	logs existants
12	Admin processes	NON

MODÉLISATION

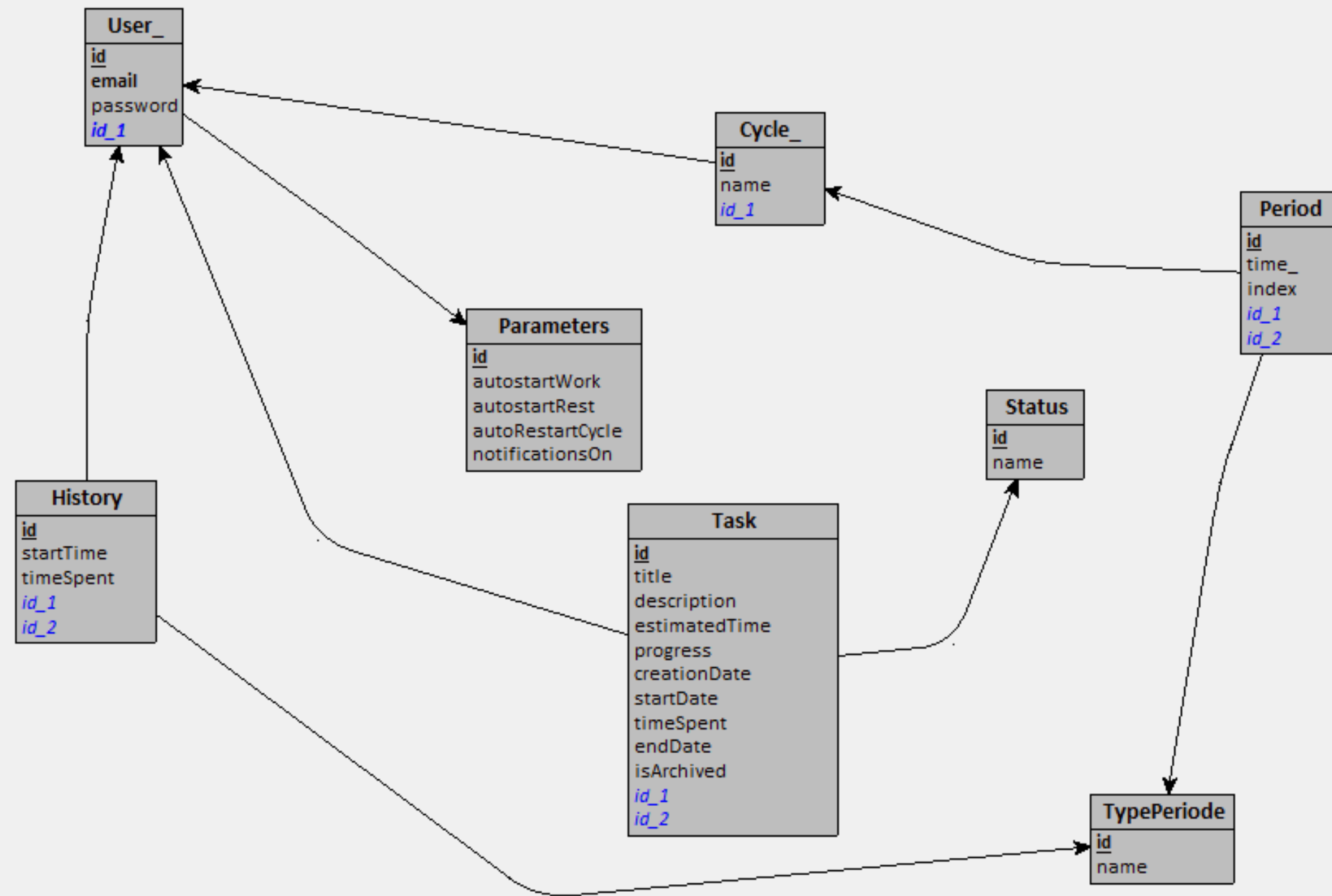
MODÉLISATION

MCD



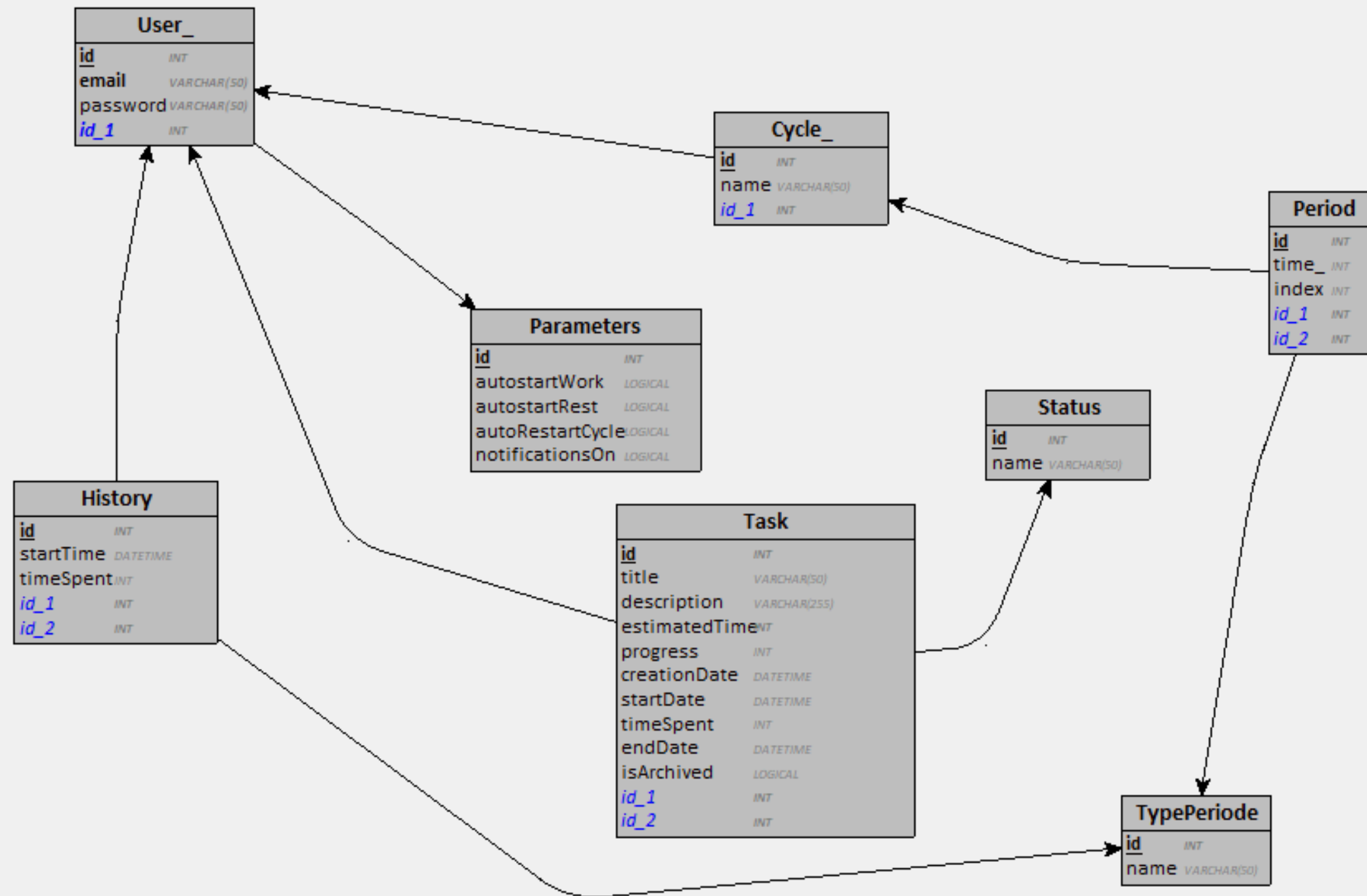
MODÉLISATION

MLD



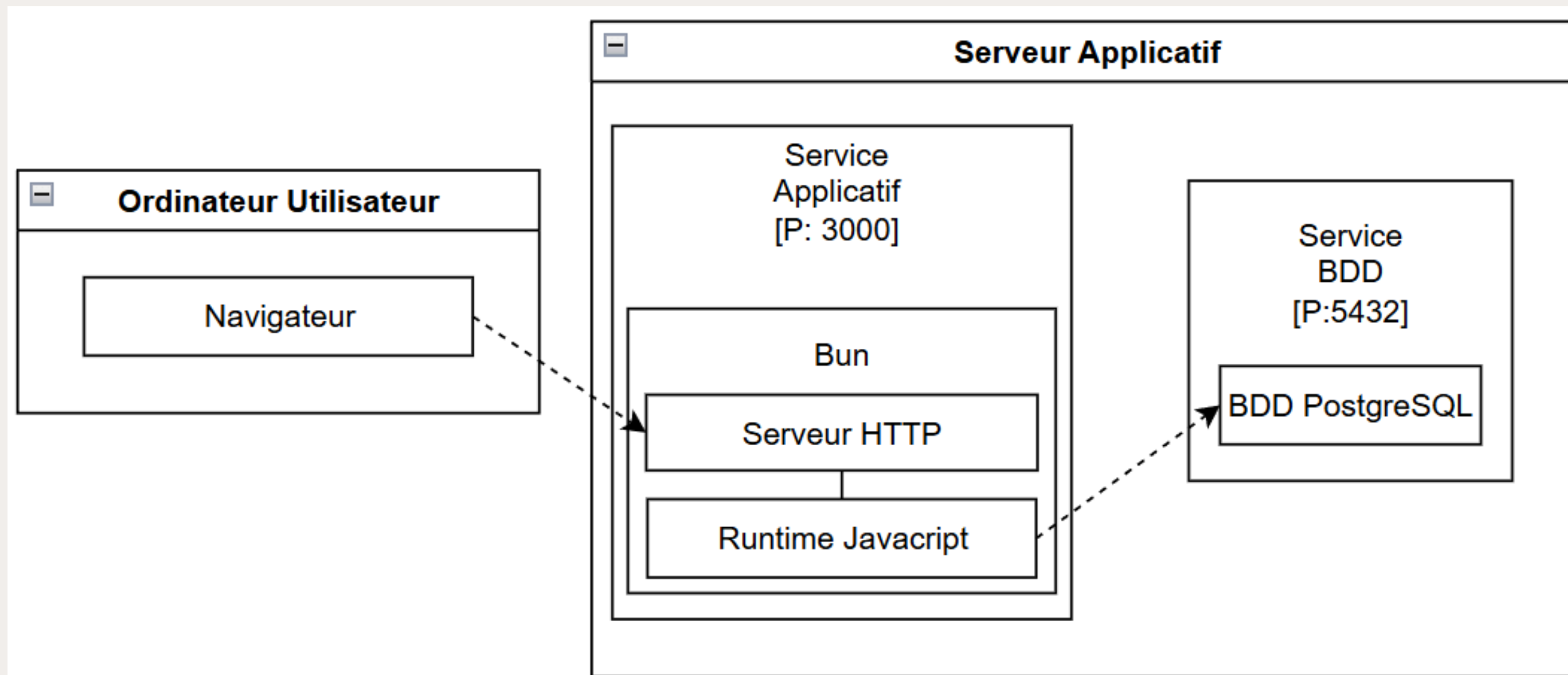
MODÉLISATION

MPD



MODÉLISATION

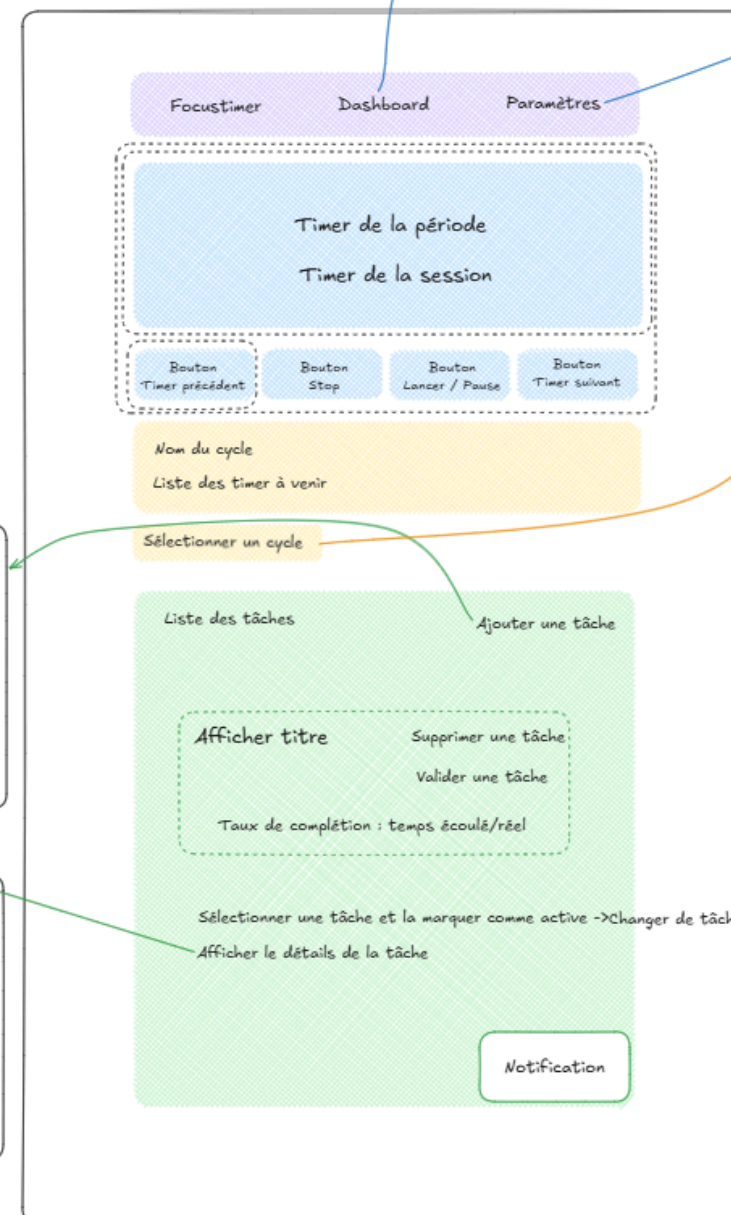
ARCHITECTURE PHYSIQUE



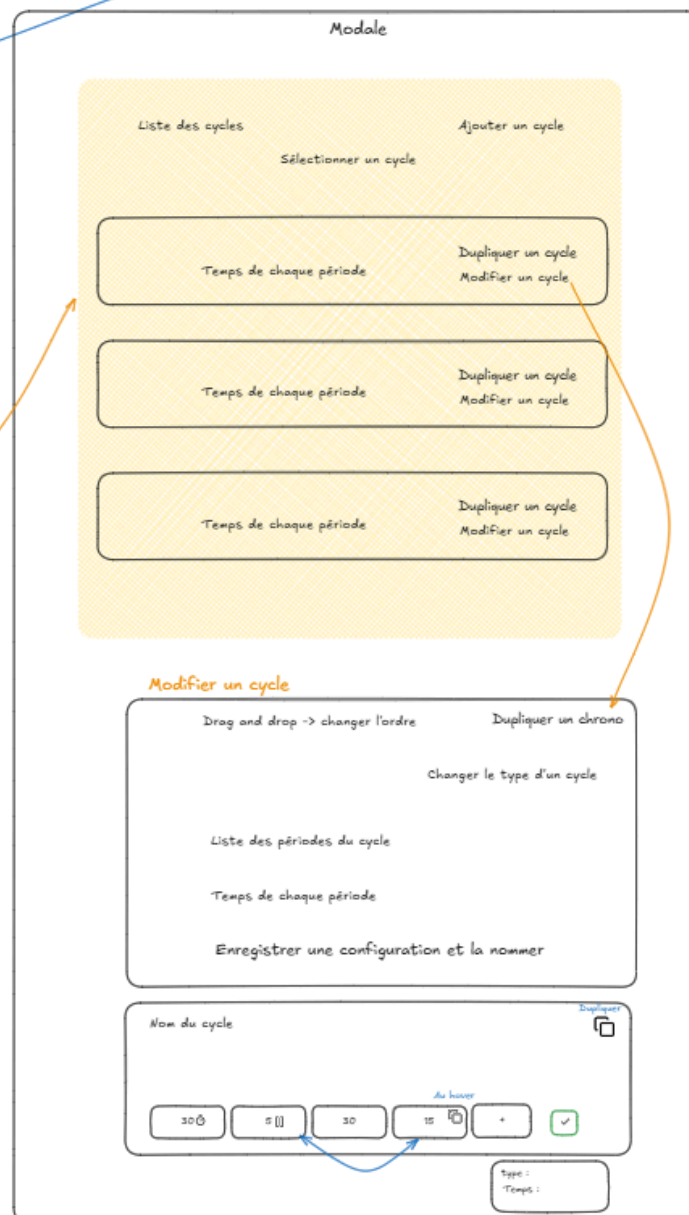
WIREFRAME

ZONING

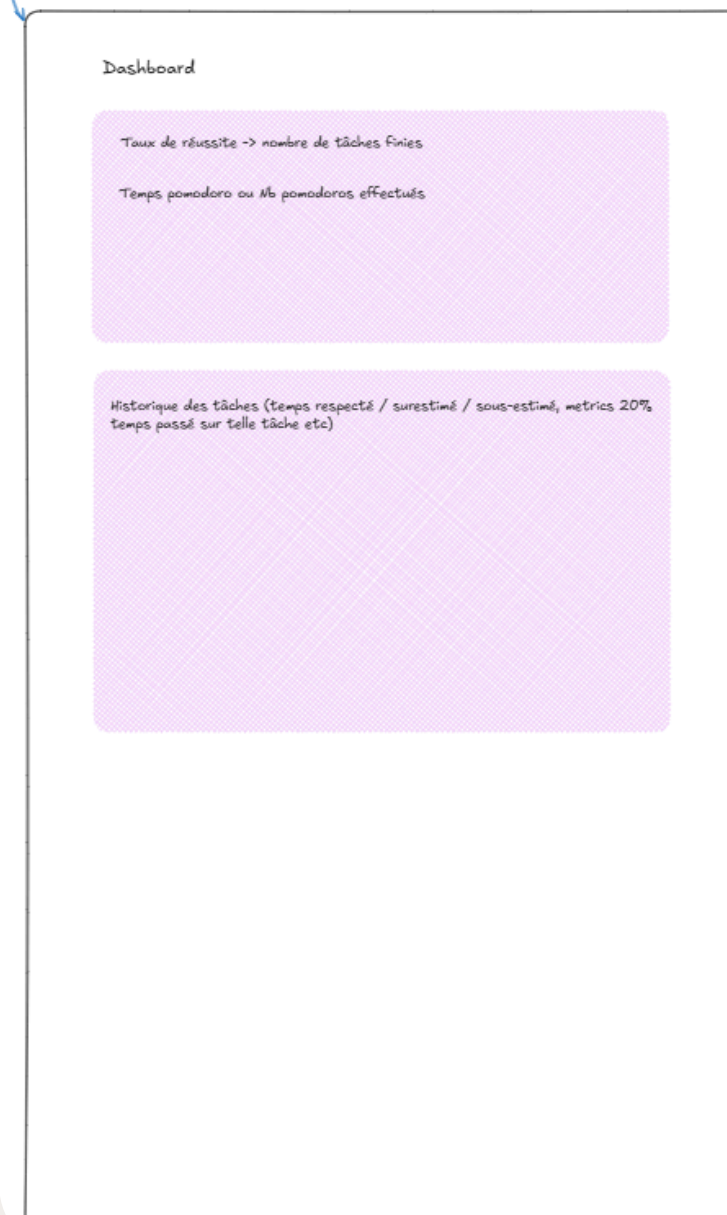
Landing



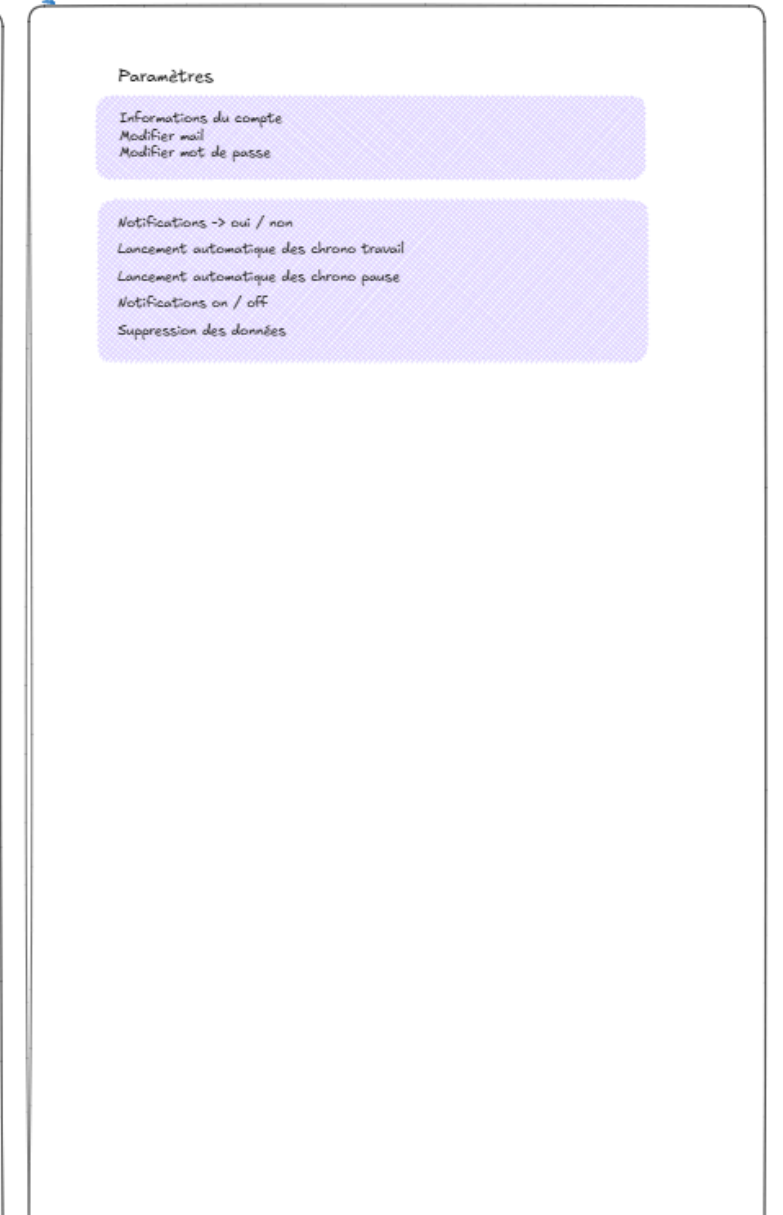
Cycle > Modale



Dashboard



Paramètres



DÉTAILS

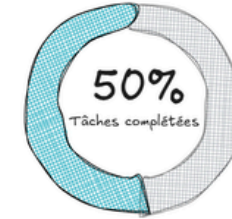
DASHBOARD

Dashboard

actuel
Complétion de la semaine

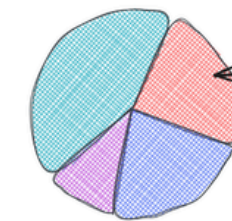
jour semaine mois 

liste des tâches sélectionnée ? Cycle ?



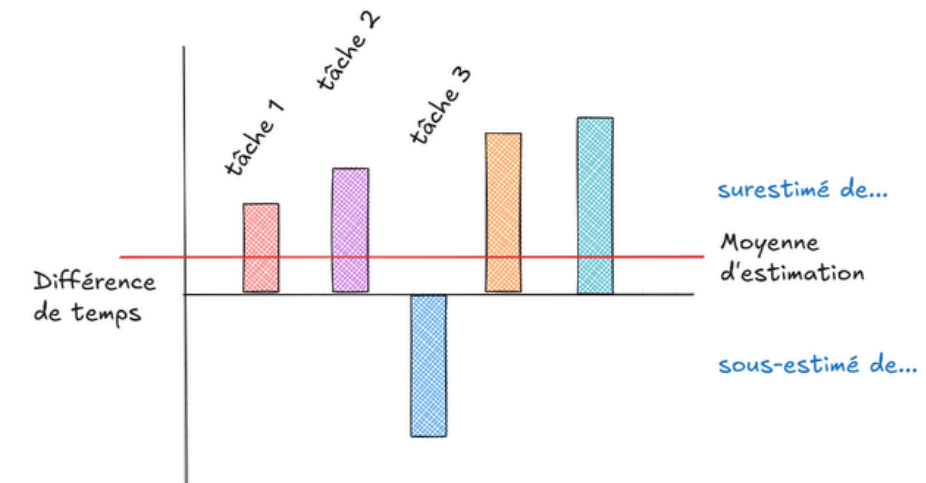
70 h de travail au total 30 h de pause au total
- 80 h de travail estimé
10 h de travail économisé

Temps passé sur telle tâche



tâche 1
2 h / 2h30

Temps moyen d'une tâche :
Temps maximum sur une tâche :
Temps minimum sur une tâche :



Calendrier

Lundi	Mardi	Mercredi	Jeudi	Vendredi	Samedi	Dimanche
travail pause						
travail pause travail pause						

CHARTRE GRAPHIQUE

0076FF Crayola Blue	2DB5FF Fresh Sky	2A2E45 Space Indigo	FEFEFE White	EBEBEB Platinum	FDCA40 Golden Pollen	F79824 Carrot Orange
------------------------	---------------------	------------------------	-----------------	--------------------	-------------------------	-------------------------

Timer

small timer

H1 - Titre

H2 - Sous-titre

H3 - Sous-titre

Body - Lorem ipsum dolor sit amet, consectetur adipiscing elit. Pellentesque nec ipsum aliquet, egestas leo bibendum, interdum lectus. Suspendisse id sapien id diam lacinia elementum ac sit amet magna. Nam porttitor semper purus, eget malesuada nisl porta at. Aenean vel mauris in dui cursus tristique vel sit amet justo. Integer ullamcorper nisi non ligula dapibus ultrices. Nunc nec urna quis nulla congue sollicitudin non ac risus. In semper feugiat dolor, eu cursus elit vehicula lobortis. Duis convallis eros non odio molestie pretium. Aliquam pretium sem risus, et efficitur tortor lacinia eu. Duis tempus sed nunc in tempus. Mauris hendrerit laoreet enim, at vestibulum lectus sodales quis.

🕒 25:00

⏸ 25:00

⏸ 25:00

🕒 25:00

25:00 Titre de la tâche ✎



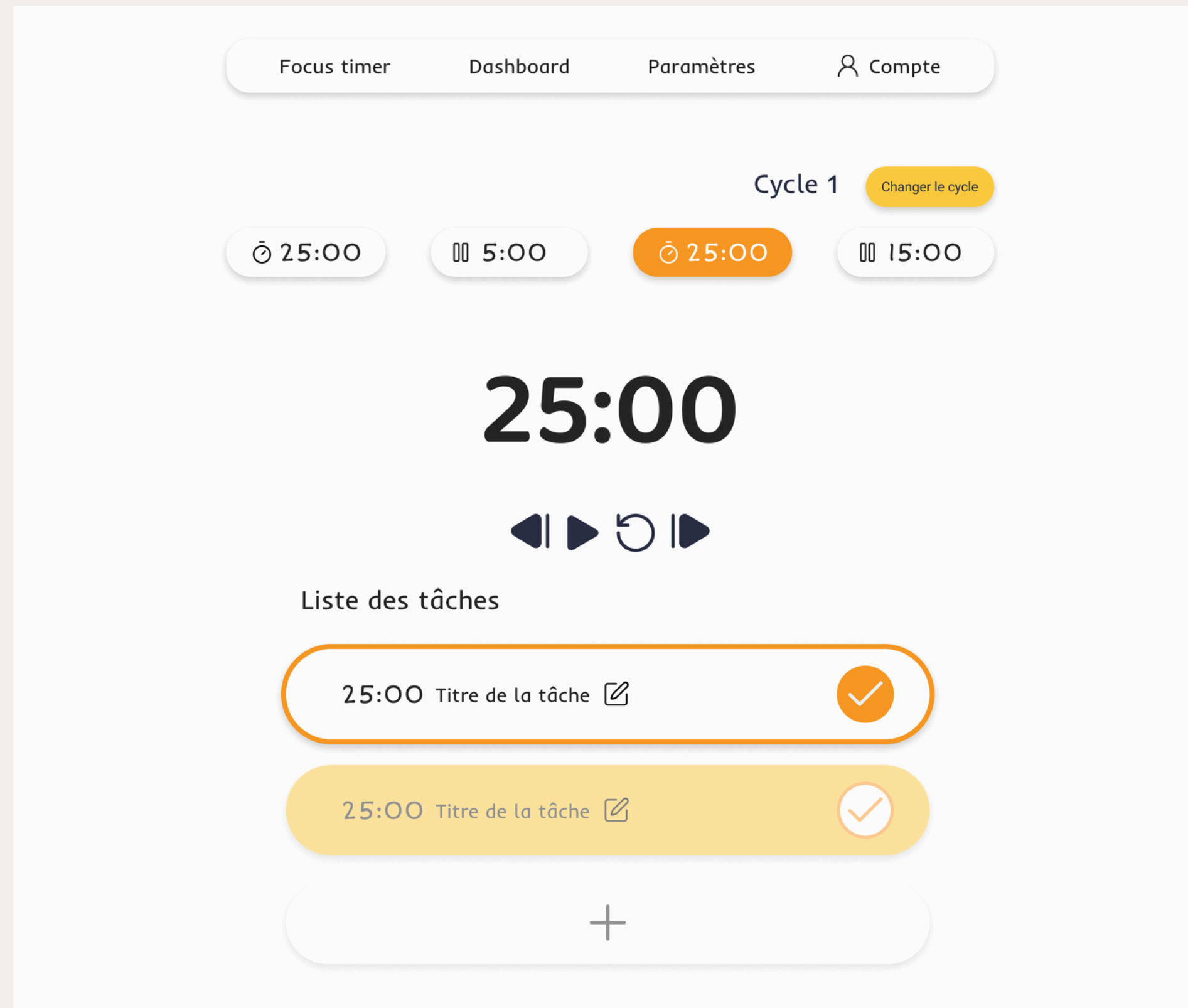
25:00 Titre de la tâche ✎
25:00



25:00 Titre de la tâche ✎



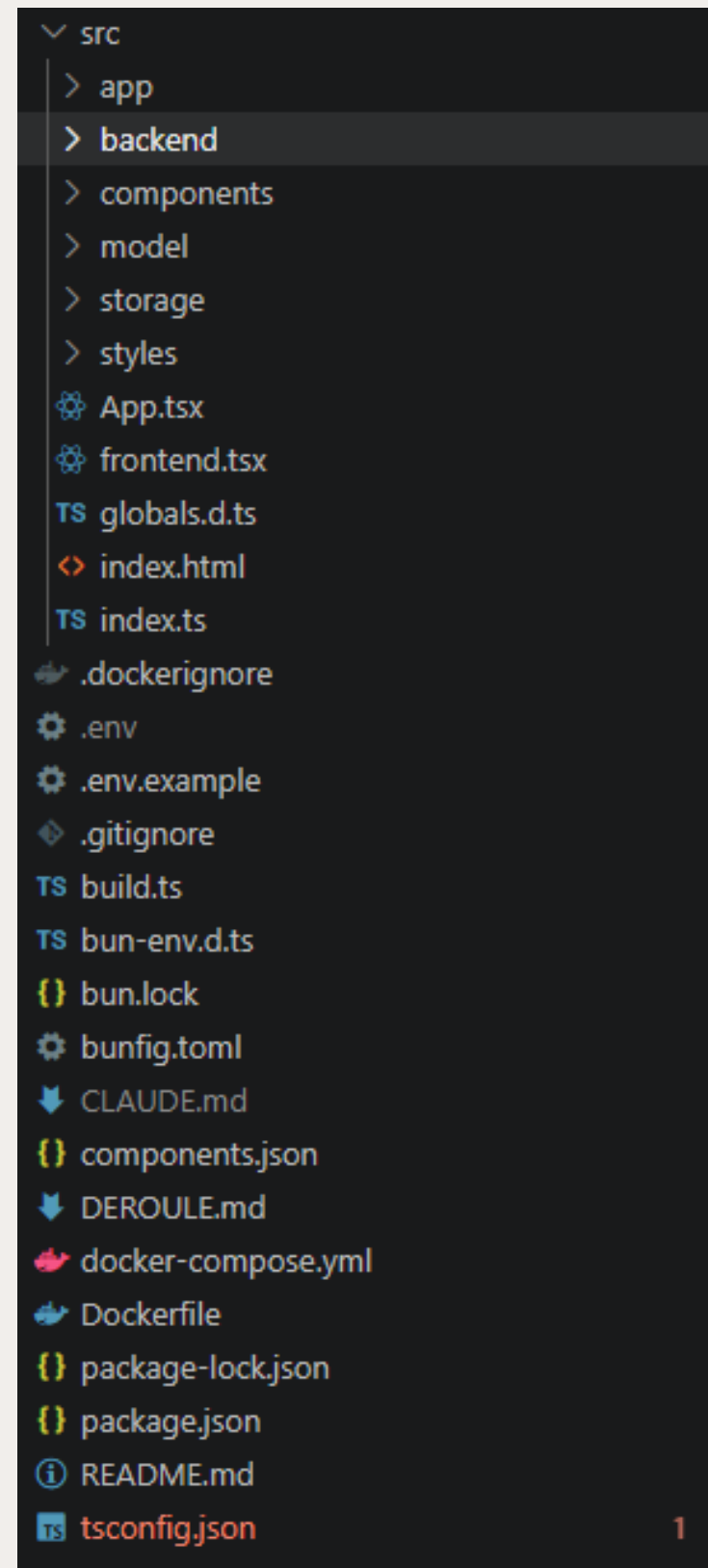
MAQUETTE



ARCHITECTURE

ARCHITECTURE

Structure des dossiers



- app → Dossier Frontend
- backend → Api / Base de donnée
- components → composant Réact
- storage → connection entre le back et front

ARCHITECTURE

Frontend

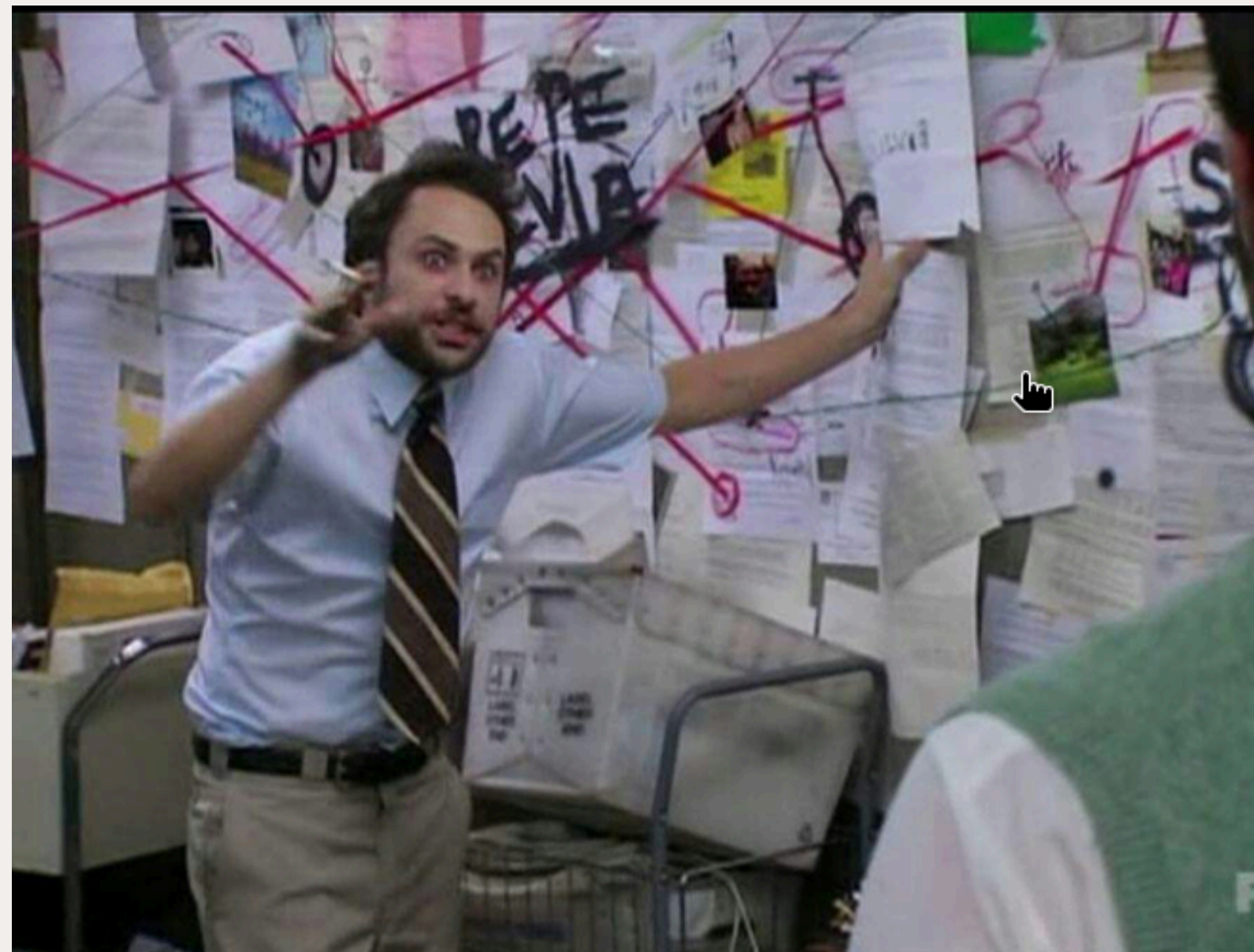
- Interface
- Navigation entre écrans (routing)
- Communication avec l'API (appels HTTP)
- Gestion de l'état de l'app

Backend

- Exposition de l'API (routes)
- Validation des entrées
- Logique métier
- Accès à la base de données
- Documentation API

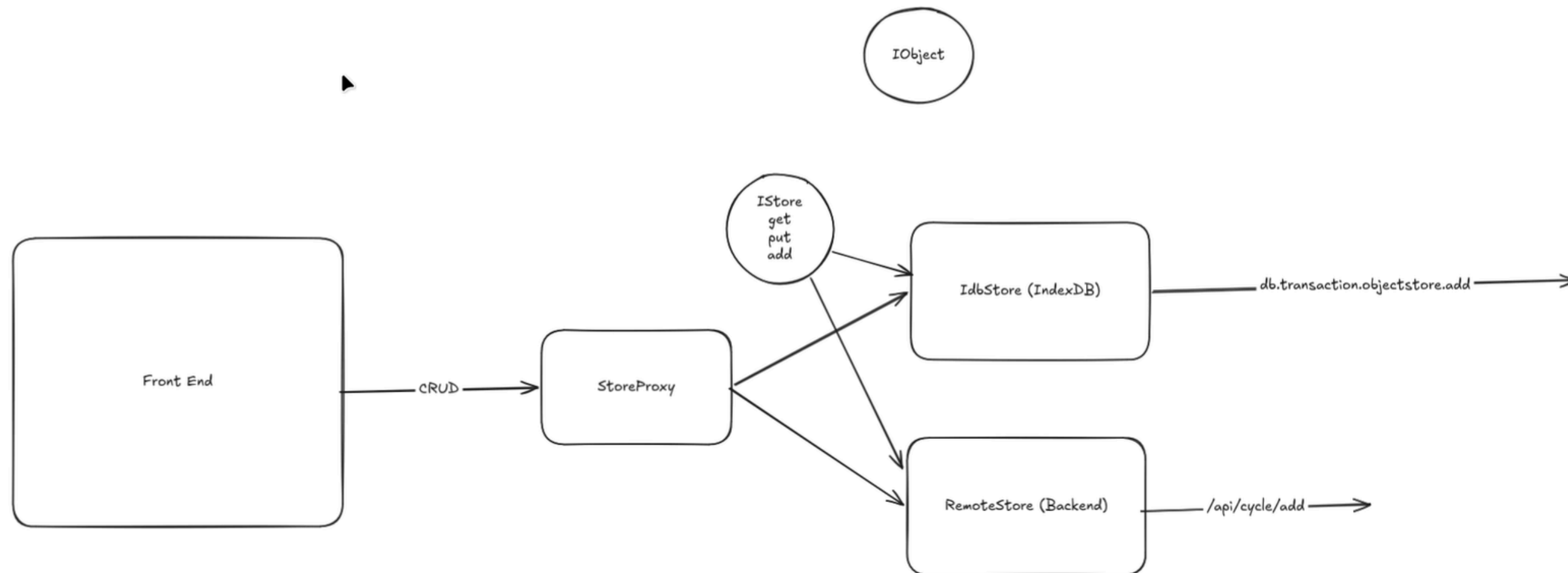
ARCHITECTURE

STOCKAGE HYBRIDE



ARCHITECTURE

STOCKAGE HYBRIDE




```

}

async getAllTasks(user_id: number): Promise<Task[]> {
    return this.request<Task[]>(`/tasks?user_id=${user_id}`, {
        method: "GET",
    });
}

async updateTaskTitle(task: Task): Promise<void> {
    await this.request<void>(`/tasks/${task.id}/title`, {
        method: "PATCH",
        body: JSON.stringify({ title: task.title }),
    });
}

async updateTaskDescription(task: Task): Promise<void> {
    await this.request<void>(`/tasks/${task.id}/description`, {
        method: "PATCH",
        body: JSON.stringify({ description: task.description }),
    });
}

async startTask(task: Task): Promise<void> {
    await this.request<void>(`/tasks/${task.id}/start`, {
        method: "POST",
        body: JSON.stringify({ startDate: task.startDate }),
    });
}

async progressTask(task: Task): Promise<void> {
    await this.request<void>(`/tasks/${task.id}/progress`, {
        method: "PUT",
        body: JSON.stringify({ timeSpent: task.timeSpent, status: task.status }),
    });
}

async endTask(task: Task): Promise<void> {
    await this.request<void>(`/tasks/${task.id}/end`, {
        method: "POST",
        body: JSON.stringify({ endDate: task.endDate, status: task.status }),
    });
}

```

Filter symbols

R

class RemoteStore

func constructor

func request

func createTask

func getAllTasks

func updateTaskTitle

func updateTaskDescription

func startTask

func progressTask

func endTask

func deleteTask

func createCycle

func getAllCycles

func getCycle

func updateCycleName

func deleteCycle

func createPeriod

func updatePeriodTime

func updatePeriodIndex

func deletePeriod

func mock


```

function getData() {
  // open a read/write db transaction, ready for retrieving the data
  const transaction = db.transaction(["todoList"], "readwrite");

  // report on the success of the transaction completing, when everything is
  done
  transaction.oncomplete = (event) => {
    note.appendChild(document.createElement("li")).textContent =
      "Transaction completed.";
  };

  transaction.onerror = (event) => {
    note.appendChild(document.createElement("li")).textContent =
      `Transaction not opened due to error: ${transaction.error}`;
  };

  // create an object store on the transaction
  const objectStore = transaction.objectStore("todoList");

  // Make a request to get a record by key from the object store
  const objectStoreRequest = objectStore.get("Walk dog");

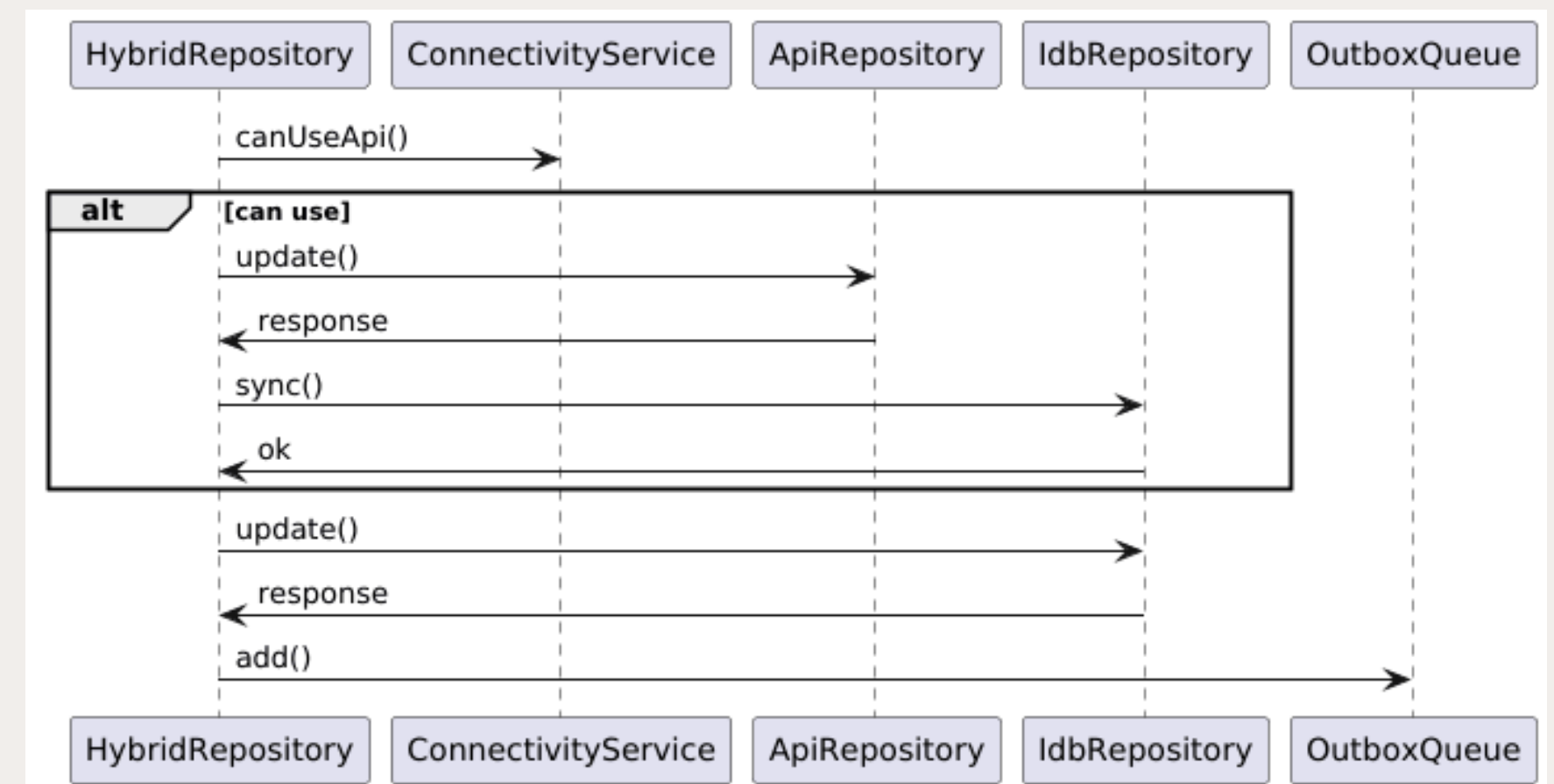
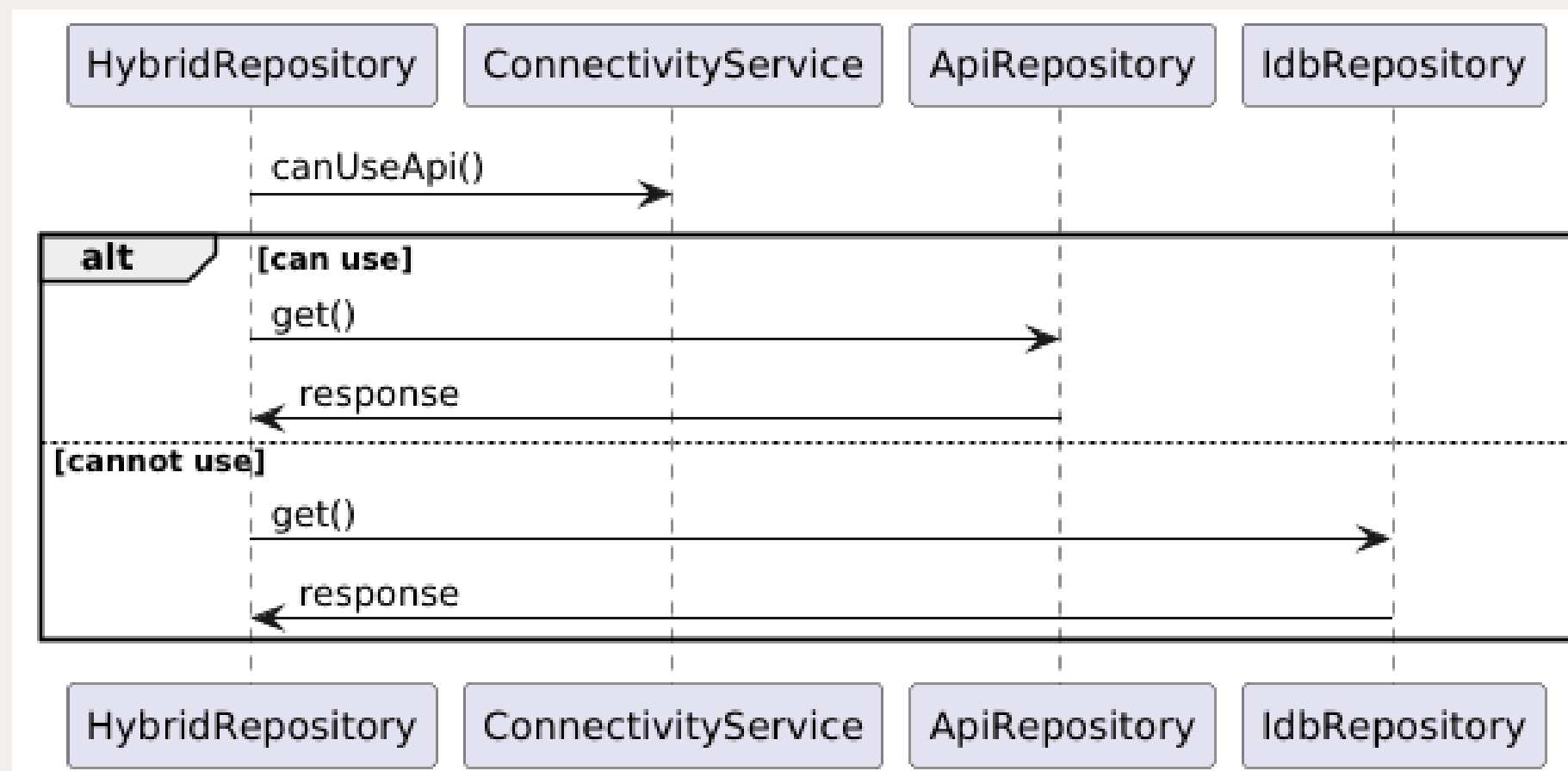
  objectStoreRequest.onsuccess = (event) => {
    // report the success of our request
    note.appendChild(document.createElement("li")).textContent =
      "Request successful.";

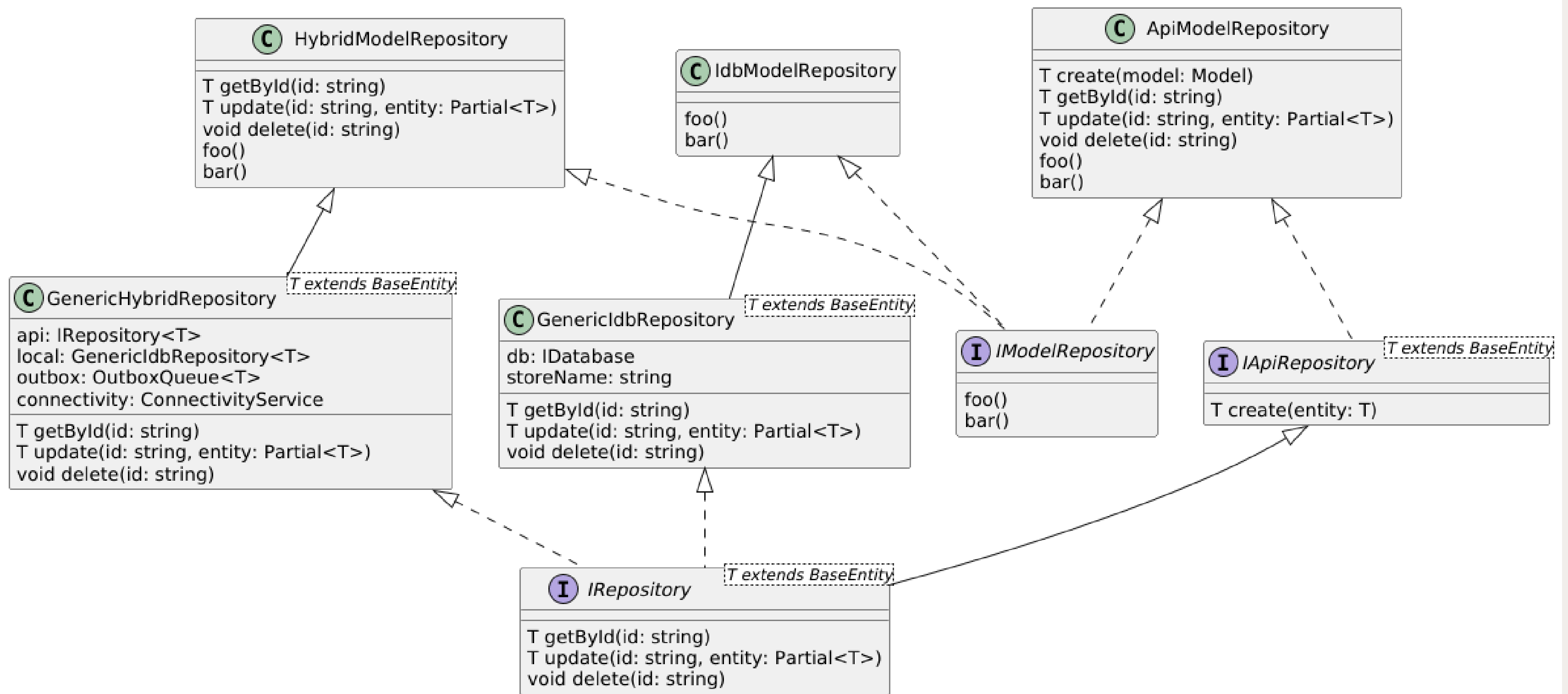
    const myRecord = objectStoreRequest.result;
  };
}

```


ARCHITECTURE

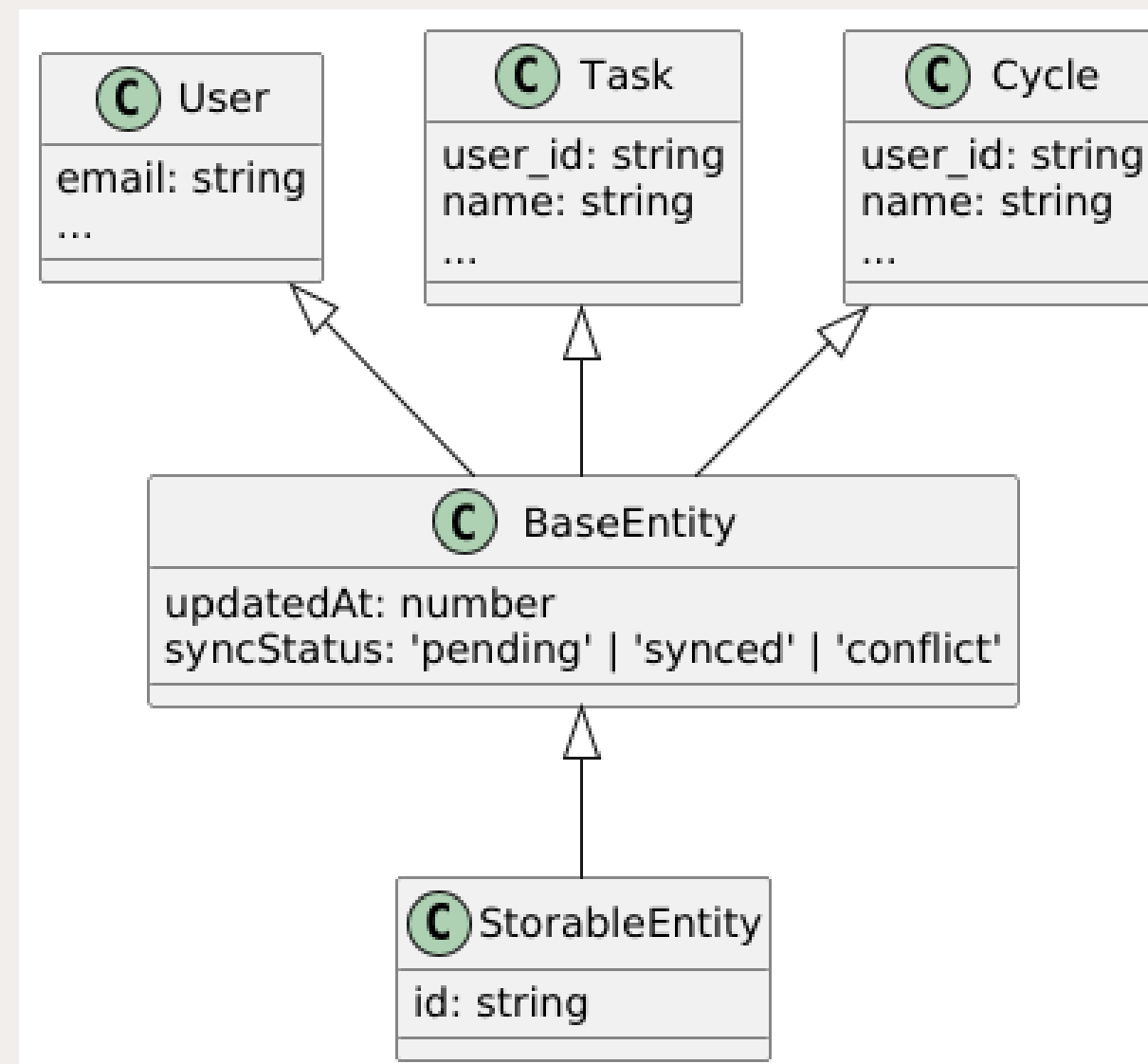
FLUX MODIFICATION HYBRIDE





ARCHITECTURE

ENTITÉE



DÉMONSTRATION

**Un grand merci
POUR VOTRE ATTENTION !**